

UNIVERSIDAD DE GUADALAJARA

Centro Universitario de Ciencias Exactas e Ingenierías

PROYECTO IoT

Aula Inteligente – Control de Temperatura y Ventilación

Asignatura:

Programación para Internet (IL-365)

Alumno:

Ragknos Demian Fernández Agraz Rodríguez – 224786978

Docente:

MICHEL EMANUEL LOPEZ FRANCO

Fecha de Entrega:

Mayo 2026

Índice de Contenidos

Índice de Contenidos.....	2
1. Objetivo General	4
2. Descripción General.....	4
3. Introducción.....	5
3.1 Planteamiento del Problema	5
3.2 Solución Propuesta.....	5
4. Análisis de Arquitectura del Sistema	6
4.1 Arquitectura de la ESP32 (Von Neumann aplicada).....	6
4.2 Arquitectura de Software del Proyecto.....	6
4.3 Arquitectura de Comunicaciones y Exposición con Ngrok	7
5. Materiales y Requerimientos del Hardware	8
5.1 Sensores	8
5.1.1 Sensor MQ-135 – Calidad del Aire / CO ₂	8
5.1.2 Sensor DHT11 – Temperatura y Humedad	8
5.2 Actuadores	9
5.2.1 Servo Motor SG90 – Control de Ventanas.....	9
5.2.2 Ventilador DC 12V – Renovación de Aire.....	9
5.2.3 Pantalla LCD 16×2 con I2C PCF8574	10
5.3 Módulo Relay de 1 Canal.....	10
5.4 Expansión Futura: Calentador PTC 110V	10
5.5 Resumen de Conexiones de Hardware	11
5.6 Plano del Modelo Físico (Maqueta).....	12
6. Base de Datos – Estructura JSON	13
6.1 Esquema de Campos del JSON	13
6.2 Ejemplo de Registro JSON (datos_ia.json)	15
6.3 Historial FIFO en Memoria RAM	16
7. Integración con Inteligencia Artificial (ChatGPT)	17
7.1 Arquitectura de la Integración	17
7.2 Prompt Embebido en el JSON	17
7.3 Ejemplo de Análisis Generado por ChatGPT	17
8. Interfaz Web de Control.....	19
8.1 Descripción de la Interfaz	19
8.2 Funcionalidades de la Interfaz	19
8.3 Endpoints HTTP de la API REST Embebida	19

8.4 Exposición Pública con Ngrok.....	20
9. Código Fuente.....	21
9.1 config.py – Configuración Central	21
9.2 boot.py – Arranque Seguro	21
9.3 actuadores.py – Control de Actuadores	22
9.4 sensores.py – Lectura de Sensores	23
9.5 sistema.py – Núcleo del Sistema	24
9.6 json_ia.py – Generador de JSON para IA	24
9.7 servidor_http.py – Servidor HTTP Embebido	25
9.8 main.py – Punto de Entrada.....	26
10. Pruebas y Validación.....	27
10.1 Pruebas Funcionales por Componente	27
10.2 Validación del Modelo JSON para IA	28
11. Documentación Técnica Complementaria	29
11.1 Configuración e Instalación.....	29
11.2 Guía de Uso de la Interfaz Web	29
11.3 Consideraciones de Seguridad	29
11.4 Posibles Expansiones Futuras	29
12. Referencias	31

1. Objetivo General

Diseñar e implementar un sistema de monitoreo y control ambiental para aulas universitarias utilizando el microcontrolador ESP32 con MicroPython, integrando sensores de temperatura, humedad y calidad del aire junto con actuadores (ventilador y ventanas automáticas), una interfaz web accesible desde cualquier dispositivo en red, un mecanismo de almacenamiento de datos en formato JSON estructurado para análisis con inteligencia artificial (ChatGPT u otra IA compatible), y exposición pública de la interfaz mediante túnel Ngrok; con el fin de comprender y aplicar de forma práctica los conceptos de arquitectura de computadoras, comunicación inalámbrica IoT, estructuras de datos y programación de sistemas embebidos.

2. Descripción General

El proyecto Aula Inteligente es un prototipo funcional basado en la ESP32 que integra sensores ambientales, actuadores físicos, un servidor HTTP embebido, una interfaz web de control y un módulo de exportación de datos en formato JSON diseñado para ser analizado directamente por modelos de lenguaje como ChatGPT sin necesidad de preprocesamiento adicional.

El sistema opera en dos modos:

- Modo Manual: el usuario controla cada actuador individualmente desde la interfaz web.
- Modo Automático: la ESP32 toma decisiones autónomas basadas en las lecturas de los sensores siguiendo umbrales configurados en config.py.

La interfaz web es servida directamente desde la ESP32 en el puerto 80 y expuesta públicamente a través de un túnel Ngrok, lo que permite acceder al sistema desde cualquier red sin configuración adicional de router.

3. Introducción

3.1 Planteamiento del Problema

El cambio climático y las condiciones ambientales dentro de los salones de clase del CUCEI representan un factor crítico que incide directamente en el rendimiento académico, la concentración y el bienestar de los estudiantes. Temperaturas extremas, niveles elevados de CO₂ y humedad inadecuada son condiciones frecuentes en el plantel durante los meses de calor.

A nivel cognitivo, la temperatura elevada reduce la memoria de trabajo y la atención; el exceso de CO₂ causa somnolencia y disminuye la velocidad de procesamiento mental; mientras que la incomodidad física derivada del calor o el frío extremo interrumpe la concentración y dificulta la escritura de apuntes.

3.2 Solución Propuesta

Se propone el Proyecto 10: un sistema IoT basado en ESP32 capaz de monitorear temperatura, humedad y calidad del aire en tiempo real, y de actuar automáticamente sobre el ambiente del aula mediante un ventilador y ventanas automatizadas. El sistema exporta todos sus datos en un JSON estructurado con descripciones en español, listo para ser analizado por ChatGPT u otra inteligencia artificial, cerrando el ciclo de monitoreo → análisis → recomendación → acción.

4. Análisis de Arquitectura del Sistema

4.1 Arquitectura de la ESP32 (Von Neumann aplicada)

El microcontrolador ESP32 es una computadora completa en un solo chip cuya organización interna sigue el modelo de Von Neumann, donde CPU, memoria y periféricos comparten un bus de datos común.

Unidad	Equivalente PC	Descripción en el Proyecto
CPU Xtensa Dual-Core LX6	Procesador Intel/AMD/ARM	Ejecuta MicroPython, el servidor HTTP, la lógica de sensores y actuadores simultáneamente en dos núcleos.
ULP Co-processor	Procesador secundario de bajo consumo	Disponible para tareas mínimas durante modos de bajo consumo.
SRAM 520 KB	RAM (memoria volátil)	Almacena variables, stack de ejecución, buffers HTTP y el JSON de IA en tiempo real.
Flash Externa 4 MB	SSD / disco duro	Almacena el firmware MicroPython, todos los archivos .py del proyecto y el archivo datos_ia.json.
ROM 448 KB	BIOS / UEFI	Contiene el bootloader de fábrica y las rutinas de inicio de hardware.
GPIO (34 pines)	Puertos USB / PCI-e	Conectan sensores DHT11, MQ-135, relay del ventilador, servo SG90 y pantalla LCD I2C.
WiFi 802.11 b/g/n	Tarjeta de red NIC	Permite la comunicación inalámbrica con la red local y el túnel Ngrok.
ADC 12 bits (18 canales)	Conversor A/D	Convierte la señal analógica del MQ-135 (0–3.3 V) en valor digital 0–4095.
I2C / SPI / UART	Bus del sistema	Comunican la pantalla LCD I2C y otros periféricos con la CPU.
PWM (16 canales)	Controlador de dispositivos	Genera la señal de control de posición del servo motor SG90.

4.2 Arquitectura de Software del Proyecto

El software está organizado en módulos MicroPython con responsabilidades claramente separadas:

Módulo	Función principal
config.py	Configuración centralizada: pines GPIO, credenciales WiFi, umbrales de temperatura y gas, parámetros del servo.
boot.py	Arranque seguro: inicializa todos los actuadores en estado apagado antes de ejecutar main.py.
main.py	Punto de entrada: orquesta WiFi, servidor HTTP y el loop principal en modo manual o automático.
sistema.py	Núcleo del sistema: coordina sensores, actuadores, historial y lógica de modos.
sensores.py	Lectura de DHT11 (temperatura, humedad, índice de calor, punto de rocío) y MQ-135 (ADC, voltaje, CO ₂ estimado).
actuadores.py	Control del relay del ventilador (Pin 26) y del servo motor SG90 (Pin 18), con paro de emergencia.
servidor_http.py	Servidor HTTP de un solo hilo con envío de HTML fragmentado para no saturar la RAM de la ESP32.
pagina_web.py	Interfaz web completa en HTML/CSS/JS servida por el servidor HTTP embebido.
historial.py	Cola FIFO de hasta 10 registros de escaneo en memoria RAM.
json_ia.py	Generador del archivo datos_ia.json con descripciones en español para análisis por IA.
wifi_manager.py	Gestor de conexión WiFi con escaneo de redes y múltiples intentos de conexión.
lcd_i2c.py	Driver completo para pantalla LCD 16×2 con adaptador I2C PCF8574.

4.3 Arquitectura de Comunicaciones y Exposición con Ngrok

La comunicación del sistema sigue el siguiente flujo:

- La ESP32 conecta a la red WiFi local (INFINITUM5DB2 o iCUCEI) y obtiene una dirección IP local.
- El servidor HTTP embebido escucha en el puerto 80 de esa IP local.
- Un servicio Ngrok corre en una computadora de la misma red y crea un túnel HTTPS desde un subdominio público de Ngrok hacia la IP local de la ESP32.
- Cualquier dispositivo en Internet puede acceder a la interfaz web usando la URL pública de Ngrok (p. ej. <https://abc123.ngrok.io>).
- Las peticiones JSON de la interfaz web (fetch()) viajan de vuelta a la ESP32 a través del mismo túnel.
- Al ejecutar un escaneo, la ESP32 actualiza datos_ia.json y lo sirve en /datos_ia para descarga y análisis con ChatGPT.

5. Materiales y Requerimientos del Hardware

5.1 Sensores

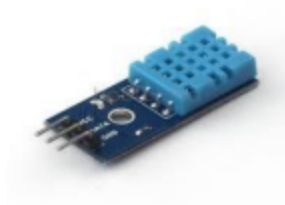
5.1.1 Sensor MQ-135 – Calidad del Aire / CO₂



Parámetro	Valor
Voltaje de operación	5 V DC
Salida analógica (AO)	0 – 3.3 V (leída por ADC 12-bit de ESP32, pin 34)
Salida digital (DO)	TTL activo en bajo al superar umbral (pin 35)
Gases detectados	CO ₂ , NH ₃ , NO _x , benceno, humo
Principio de operación	Variación de conductancia en material semiconductor SnO ₂
Conexión en proyecto	VCC → 5V, GND → GND, AO → GPIO 34, DO → GPIO 35

5.1.2 Sensor DHT11 – Temperatura y Humedad

Parámetro	Valor
Voltaje de operación	3.3 – 5 V DC
Rango de temperatura	0 – 50 °C (±2 °C)
Rango de humedad	20 – 90 % HR (±5 %)
Protocolo	1-Wire digital
Tiempo de muestreo	1 dato por segundo mínimo
Conexión en proyecto	VCC → 3.3V, GND → GND, DATA → GPIO 4



5.2 Actuadores

5.2.1 Servo Motor SG90 – Control de Ventanas



Parámetro	Valor
Voltaje de operación	4.8 – 6 V DC
Torque	1.8 kg·cm a 4.8 V
Rango de movimiento	0 – 180 grados
Frecuencia PWM	50 Hz (período 20 ms)
Posición 0° (cerrado)	duty_u16 = 1638 (pulso 0.5 ms)
Posición 30° (abierto)	duty_u16 = 4915 (pulso ajustado al mecanismo físico)
Conexión en proyecto	VCC → 5V, GND → GND, SIG → GPIO 18 (PWM)

5.2.2 Ventilador DC 12V – Renovación de Aire



Parámetro	Valor
Voltaje de operación	12 V DC
Control	Relay de 1 canal (lógica invertida: LOW = encendido)
Pin de relay	GPIO 26
Umbral de activación automática	Temperatura > 28 °C
Fuente de alimentación	Batería externa 3×AA / fuente DC 12V

5.2.3 Pantalla LCD 16×2 con I2C PCF8574



Parámetro	Valor
Resolución	16 columnas × 2 filas
Interfaz	I2C (2 hilos) mediante módulo PCF8574
Dirección I2C	0x3F (configurable por jumpers)
Pines SDA / SCL	GPIO 21 / GPIO 22
Función en proyecto	Muestra temperatura, CO ₂ estimado, modo y estado de actuadores

5.3 Módulo Relay de 1 Canal

Parámetro	Valor
Voltaje de control	5 V (activo en LOW desde ESP32)
Capacidad de conmutación	10 A / 250 VAC – 10 A / 30 VDC
Lógica	Invertida: relay activo cuando el pin GPIO está en 0 (LOW)
Uso	Controla la alimentación 12V del ventilador DC

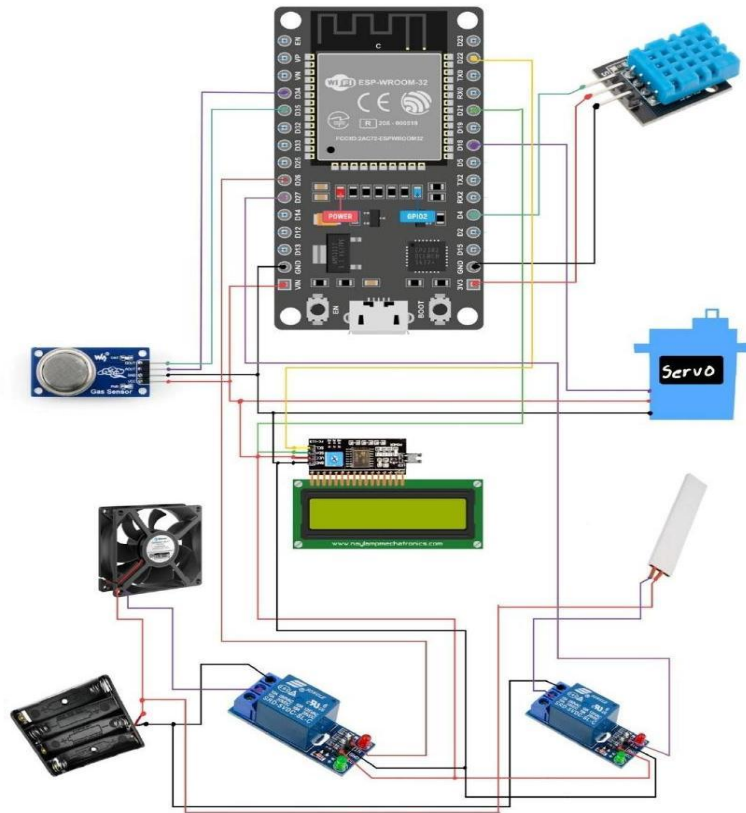
5.4 Expansión Futura: Calentador PTC 110V



Como expansión futura del proyecto se contempla integrar una resistencia cerámica PTC de 110 V para calefacción del aula durante condiciones de temperatura baja ($< 10\text{ }^{\circ}\text{C}$). El código original del proyecto (documento entregable en el reporte) ya incluye la lógica completa para un segundo relay (`PIN_RELAY2 = 27`) y la función `activar_calentador()`. En la versión actual del hardware físico no se instaló el calentador por razones de seguridad eléctrica en el prototipo; su activación está preparada en el firmware para futuras iteraciones.

5.5 Resumen de Conexiones de Hardware

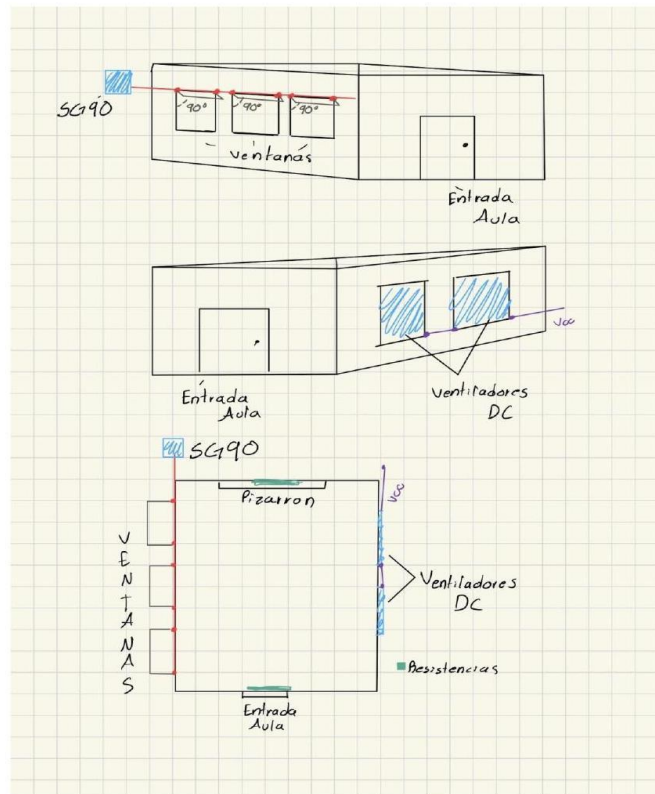
onexiones



Componente	Pin ESP32	Función
DHT11 DATA	GPIO 4	Lectura de temperatura y humedad (1-Wire)
MQ-135 AO	GPIO 34	Lectura analógica de calidad del aire (ADC 12-bit)
MQ-135 DO	GPIO 35	Alarma digital de hardware (activo en LOW)
Relay Ventilador	GPIO 26	Control del ventilador DC 12V (lógica invertida)
Servo SG90 SIG	GPIO 18	Control de posición de ventanas (PWM 50 Hz)
LCD SDA	GPIO 21	Datos I2C para pantalla LCD
LCD SCL	GPIO 22	Reloj I2C para pantalla LCD

5.6 Plano del Modelo Físico (Maqueta)

Plano



El plano muestra la distribución del aula maqueta con las ventanas en la pared lateral (controladas por el servo SG90), los ventiladores DC en la pared opuesta para flujo de aire cruzado, y la zona de la pizarra donde se concentran los sensores de temperatura y calidad del aire.

6. Base de Datos – Estructura JSON

El sistema no utiliza una base de datos relacional tradicional. En su lugar, la ESP32 genera y mantiene un archivo JSON (datos_ia.json) en su sistema de archivos LittleFS que actúa como la base de datos del sistema. Este archivo:

- Se sobrescribe en cada escaneo (registro más reciente siempre disponible).
- Contiene valores numéricos reales de los sensores y descripciones textuales en español.
- Incluye un prompt de instrucción para IA al final del documento.
- Puede descargarse desde la interfaz web mediante la ruta /datos_ia.
- Sirve como entrada directa para ChatGPT u otro modelo de lenguaje sin preprocesamiento.

6.1 Esquema de Campos del JSON

Campo / Sección	Tipo	Descripción
descripcion_documento	string	Explicación general del documento para orientar a la IA.
proyecto.nombre	string	Nombre del sistema.
proyecto.asignatura	string	Materia en la que se desarrolló.
proyecto.integrantes	string	Nombres y números de registro del equipo.
proyecto.hardware	string	Plataforma de hardware utilizada.
proyecto.sensores	string	Sensores integrados al sistema.
proyecto.actuadores	string	Actuadores controlados por la ESP32.
marca_tiempo.segundos_desde_inicio	int	Segundos transcurridos desde el encendido de la ESP32 (no RTC real).
modo_operacion.modo	string	'manual' o 'automatico'
modo_operacion.descripcion	string	Descripción textual del modo de operación actual.
temperatura_y_humedad.temperatura_celsius.valor	float	Temperatura real en grados Celsius (DHT11).

Campo / Sección	Tipo	Descripción
temperatura_y_humedad.humedad_relativa_porcentaje.valor	float	Humedad relativa en % (DHT11).
temperatura_y_humedad.indice_de_calor_celsius.valor	float	Sensación térmica calculada con fórmula de Rothfusz.
temperatura_y_humedad.punto_de_rocio_celsius.valor	float	Punto de rocío calculado con fórmula de Magnus.
temperatura_y_humedad.confort_termico.clasificacion	string	Etiqueta de confort: Confortable / Seco / Calido / Muy calido, etc.
calidad_del_aire.valor_adc_crudo.valor	int	Promedio de 10 muestras del ADC 12-bit (0–4095).
calidad_del_aire.voltaje_pin_voltios.valor	float	Voltaje real en el pin analógico del MQ-135 (0.0–3.3 V).
calidad_del_aire.desviacion_estandar_adc.valor	int	Desviación estándar de las 10 muestras (estabilidad de lectura).
calidad_del_aire.calidad_aire_porcentaje.valor	int	Porcentaje de calidad del aire (100 = limpio).
calidad_del_aire.saturacion_sensor_porcentaje.valor	int	Porcentaje de saturación del sensor MQ-135 (0 = limpio).
calidad_del_aire.co2_estimado_ppm.valor	int	Estimación de CO ₂ en ppm por interpolación sobre curva del MQ-135.
calidad_del_aire.condicion_del_aire.clasificacion	string	Óptimo / Aceptable / Regular / Deficiente / Peligroso.
calidad_del_aire.nivel_de_riesgo.valor	int	Nivel de riesgo 1 (mínimo) a 5 (peligroso).
calidad_del_aire.recomendacion_ventilacion.accion	string	Acción recomendada por el sistema.
calidad_del_aire.alarma_hardware_do.activa	bool	true si el pin DO del MQ-135 indica alarma.
estado_actuadores.ventilador_12v.encendido	bool	Estado actual del relay del ventilador.
estado_actuadores.ventanas_servo.abiertas	bool	true si el servo está en posición abierta.
estado_actuadores.paro_de_emergencia.activo	bool	true si el paro de emergencia está activo.
evaluacion_global.descripcion	string	Evaluación automática del estado general del aula.

Campo / Sección	Tipo	Descripción
instruccion_para_ia	string	Prompt embebido que instruye a la IA sobre cómo analizar el documento.

6.2 Ejemplo de Registro JSON (datos_ia.json)

A continuación se muestra un ejemplo representativo del JSON generado tras un escaneo con temperatura de 29 °C, humedad del 65 % y calidad del aire aceptable:

```
{
  "descripcion_documento": "Este documento JSON contiene los datos del ultimo
escaneo ambiental
  del sistema Aula Inteligente basado en ESP32. Esta redactado en espanol para
ser
  analizado directamente por una inteligencia artificial.",
  "proyecto": {
    "nombre": "Aula Inteligente - Control de Temperatura y Ventilacion",
    "asignatura": "Programacion Internet",
    "integrantes": "Ragknos Demian Fernandez (224786978)",
    "hardware": "ESP32 con MicroPython",
    "sensores": "DHT11 (temperatura y humedad), MQ-135 (calidad del aire)",
    "actuadores": "Ventilador 12V via Relay1 (pin 26), Ventanas via Servo SG90
(pin 18)"
  },
  "marca_tiempo": { "segundos_desde_inicio": 3840 },
  "modo_operacion": { "modo": "automatico" },
  "temperatura_y_humedad": {
    "temperatura_celsius": { "valor": 29 },
    "humedad_relativa_porcentaje": { "valor": 65 },
    "indice_de_calor_celsius": { "valor": 31.2 },
    "punto_de_rocio_celsius": { "valor": 22.1 },
    "confort_termico": { "clasificacion": "Calido" }
  },
  "calidad_del_aire": {
    "valor_adc_crudo": { "valor": 1450, "escala": "0 a 4095" },
    "voltaje_pin_voltios": { "valor": 1.168 },
    "desviacion_estandar_adc": { "valor": 12 },
    "calidad_aire_porcentaje": { "valor": 65 },
    "saturacion_sensor_porcentaje": { "valor": 35 },
    "co2_estimado_ppm": { "valor": 900, "unidad": "partes por millon (ppm)" },
    "condicion_del_aire": { "clasificacion": "Aceptable" },
    "nivel_de_riesgo": { "valor": 2, "escala": "1 (optimo) a 5 (peligroso)" },
    "recomendacion_ventilacion": { "accion": "Ventilacion moderada recomendada"
  },
  "alarma_hardware_do": { "activa": false }
},
  "estado_actuadores": {
    "ventilador_12v": { "encendido": true },
    "ventanas_servo": { "abiertas": false },
  }
}
```

```

    "paro_de_emergencia": { "activo": false }
  },
  "evaluacion_global": {
    "descripcion": "ESTADO GENERAL: ATENCION. Temperatura por encima del limite
de confort (28C).\"
  },
  "instruccion_para_ia": \"Eres un asistente de analisis ambiental. Analiza los
datos
de este JSON que corresponden al ultimo escaneo del Aula Inteligente...\"
}

```

6.3 Historial FIFO en Memoria RAM

Complementariamente al archivo JSON, el sistema mantiene en memoria RAM un historial FIFO (First In, First Out) de hasta 10 escaneos reales. Este historial se muestra en la interfaz web y se incluye en las respuestas JSON de los endpoints /escanear y /escanear_auto:

Campo	Tipo	Descripción
seg_boot	int	Marca de tiempo en segundos desde el inicio de la ESP32.
temperatura	float	Temperatura del escaneo en °C.
humedad	float	Humedad relativa en %.
indice_calor	float	Índice de calor calculado en °C.
punto_rocio	float	Punto de rocío en °C.
confort	string	Clasificación de confort térmico.
gas_adc	int	Valor ADC crudo del MQ-135 (0–4095).
gas_voltaje	float	Voltaje en pin analógico (V).
gas_pct	int	Porcentaje de calidad del aire.
gas_ppm_co2	int	CO ₂ estimado en ppm.
gas_condicion	string	Condición del aire (Óptimo, Aceptable, etc.).
gas_riesgo	int	Nivel de riesgo 1–5.
gas_alarma	bool	Estado de la alarma DO del MQ-135.

7. Integración con Inteligencia Artificial (ChatGPT)

7.1 Arquitectura de la Integración

La integración con IA no requiere una API de pago ni configuración compleja. El flujo completo es:

- La ESP32 genera datos_ia.json en cada escaneo con todos los datos del aula más el campo instruccion_para_ia.
- El usuario descarga el JSON desde la interfaz web (botón 'Descargar datos IA' → endpoint /datos_ia).
- El archivo descargado se carga directamente en ChatGPT (adjuntar archivo) o se pega su contenido en el chat.
- ChatGPT lee el campo instruccion_para_ia como su instrucción de sistema y los demás campos como el contexto de datos.
- ChatGPT produce un análisis en español con: evaluación de condiciones, identificación de problemas, validación de actuadores y recomendaciones concretas.

7.2 Prompt Embebido en el JSON

El campo instruccion_para_ia del JSON contiene el siguiente prompt de instrucción:

"Eres un asistente de análisis ambiental. Analiza los datos de este JSON que corresponden al último escaneo del Aula Inteligente. Identifica si existen condiciones fuera de rango, evalúa el confort de los ocupantes, determina si los actuadores están respondiendo correctamente a las condiciones detectadas, y proporciona recomendaciones concretas en español para mejorar el ambiente del aula."

7.3 Ejemplo de Análisis Generado por ChatGPT

A continuación se muestra un ejemplo representativo de la respuesta que ChatGPT genera al recibir el JSON de un escaneo con temperatura de 29 °C y CO₂ aceptable:

Aspecto Analizado	Resultado / Respuesta de ChatGPT
Temperatura	La temperatura de 29 °C supera el umbral de confort recomendado (28 °C). El índice de calor de 31.2 °C indica que la humedad amplifica la sensación térmica. El ventilador se activó correctamente en modo automático.
Humedad	La humedad relativa del 65 % se encuentra en el rango confortable (40–70 %). No se requiere intervención.

Aspecto Analizado	Resultado / Respuesta de ChatGPT
Calidad del Aire	El ADC de 1450 corresponde a una condición Aceptable con ~900 ppm de CO ₂ estimado, dentro del límite recomendado de 1000 ppm. No se abrieron ventanas automáticamente, lo que es correcto.
Estado de Actuadores	El ventilador está encendido apropiadamente por la temperatura alta. Las ventanas están cerradas porque el nivel de gases no supera el umbral de 2500. El paro de emergencia está inactivo.
Recomendaciones	1. Monitorear la temperatura en los próximos minutos para verificar que el ventilador logre reducirla. 2. Si la temperatura supera 31 °C o el CO ₂ supera 1000 ppm, abrir ventanas manualmente. 3. Considerar aumentar la frecuencia de escaneo a 30 segundos durante horas pico.
Evaluación Global	ESTADO: ATENCIÓN RECOMENDADA. La temperatura está marginalmente fuera del rango óptimo. El sistema está respondiendo correctamente. Se recomienda continuar monitoreando.

8. Interfaz Web de Control

8.1 Descripción de la Interfaz

La interfaz web del Aula Inteligente se sirve directamente desde la ESP32 mediante el servidor HTTP embebido. Está desarrollada en HTML5, CSS3 y JavaScript vanilla, optimizada para funcionar en las restricciones de memoria de la ESP32 (el HTML se envía en fragmentos de 512 bytes para no saturar la RAM).

8.2 Funcionalidades de la Interfaz

Sección de la Web	Función
Selector de Modo	Botones para cambiar entre Modo Manual y Modo Automático con actualización inmediata en la ESP32.
Escaneo Ambiental	Muestra temperatura, humedad, calidad del aire, valor ADC crudo y estado de alarma hardware en tiempo real.
Control de Actuadores (Manual)	Botones para encender/apagar el ventilador y abrir/cerrar las ventanas individualmente.
Panel Automático	Muestra el estado de los actuadores actualizados automáticamente en modo automático con cuenta regresiva de 60 s.
Seguridad del Sistema	Botón de PARO DE EMERGENCIA (apaga todo y bloquea el sistema) y botón de Reiniciar Circuito.
Historial FIFO	Tabla con los últimos 10 escaneos reales ordenados del más reciente al más antiguo.
Footer	Información del equipo, asignatura y fecha de entrega cargada dinámicamente desde la ESP32.

8.3 Endpoints HTTP de la API REST Embebida

Método	Ruta	Descripción
GET	/	Sirve la página web completa (HTML fragmentado).
GET	/estado	Retorna JSON con el estado completo del sistema.
POST	/ventilador	Enciende o apaga el ventilador. Body: {"accion": "encender" "apagar"}
POST	/ventanas	Abre o cierra las ventanas. Body: {"accion": "abrir" "cerrar"}
POST	/escanear	Ejecuta un escaneo manual y retorna datos + historial.
POST	/escanear_auto	Ejecuta un escaneo automático (con lógica de actuadores) y retorna datos.
POST	/paro	Activa el paro de emergencia.

Método	Ruta	Descripción
POST	/reiniciar	Reinicia el circuito desde el paro de emergencia.
POST	/modo	Cambia el modo de operación. Body: {"modo": "manual" "automatico"}
GET	/datos_ia	Descarga el archivo datos_ia.json del último escaneo.

8.4 Exposición Pública con Ngrok

Para que la interfaz web sea accesible desde Internet sin modificar la configuración del router universitario, se utiliza Ngrok:

- Se instala el cliente Ngrok en una computadora conectada a la misma red WiFi que la ESP32.
- Se ejecuta: `ngrok http [IP_DE_ESP32]:80`
- Ngrok crea un túnel HTTPS con URL pública (ej. `https://abc123.ngrok-free.app`).
- La URL pública se comparte con los evaluadores para acceso remoto a la interfaz.
- Ngrok actúa como proxy inverso: recibe las peticiones HTTPS externas, las reenvía como HTTP a la ESP32 y retorna las respuestas al cliente original.

9. Código Fuente

A continuación, se presenta el código fuente completo de cada módulo del proyecto. Todos los archivos están escritos en MicroPython y diseñados para ejecutarse en la ESP32.

9.1 config.py – Configuración Central

Define todas las constantes del sistema: credenciales WiFi, pines GPIO, umbrales de temperatura y gas, parámetros del servo e información del equipo.

```
# config.py - Configuración central del Aula Inteligente
REDES_WIFI = [
    {"ssid": "INFINITUM5DB2", "password": "t4UNUWMK34"},
    {"ssid": "iCUCEI", "password": None},
]
TIEMPO_ESPERA_WIFI = 8
PIN_DHT11 = 4 # Sensor temperatura y humedad
PIN_MQ135_A = 34 # MQ-135 salida analogica
PIN_MQ135_D = 35 # MQ-135 salida digital
PIN_RELAY1 = 26 # Relay 1 -> Ventilador 12V
PIN_SERVO = 18 # Servo motor -> Ventanas
PIN_LCD_SDA = 21 # LCD I2C - Datos
PIN_LCD_SCL = 22 # LCD I2C - Reloj
LCD_ADDR = 0x3F
SERVO_FREQ = 50
SERVO_CERRADO = 1638 # 0 grados
SERVO_ABIERTO = 4915 # 30 grados (ajustado al mecanismo)
TEMP_MAX_VENTILADOR = 28 # C: activar ventilador si supera esto
UMBRAL_GAS_PELIGROSO = 2500 # ADC: abrir ventanas si supera esto
MAX_HISTORIAL = 10
INTERVALO_ESCANEO = 60
```

9.2 boot.py – Arranque Seguro

Apaga todos los actuadores al encender la ESP32 para evitar estados peligrosos o indefinidos en el hardware.

```
# boot.py - Arranque seguro del sistema Aula Inteligente
from machine import Pin, PWM
import time
PIN_RELAY1 = 26
PIN_SERVO = 18
relay1 = Pin(PIN_RELAY1, Pin.OUT)
relay1.value(1) # HIGH = relay desactivado (logica invertida)
servo_pwm = PWM(Pin(PIN_SERVO), freq=50)
servo_pwm.duty_u16(1638) # Angulo 0 grados = ventanas cerradas
time.sleep(0.5)
```

```
servo_pwm.deinit()  
print("[BOOT] Todos los actuadores apagados de forma segura.")
```

9.3 actuadores.py – Control de Actuadores

Gestiona el ventilador (via relay en Pin 26) y las ventanas (servo SG90 en Pin 18). Incluye paro de emergencia con bloqueo y función de reinicio de circuito.

```
# actuadores.py - Control de actuadores del Aula Inteligente  
from machine import Pin, PWM  
from config import PIN_RELAY1, PIN_SERVO, SERVO_FREQ, SERVO_CERRADO,  
SERVO_ABIERTO  
  
class ControladorActuadores:  
    def __init__(self):  
        self._relay1 = Pin(PIN_RELAY1, Pin.OUT, value=1)  
        self._servo_pwm = PWM(Pin(PIN_SERVO), freq=SERVO_FREQ)  
        self._servo_pwm.duty_u16(SERVO_CERRADO)  
        self._ventilador_on = False  
        self._ventanas_open = False  
        self._paro_activo = False  
  
    def activar_ventilador(self) -> bool:  
        if self._paro_activo: return False  
        self._relay1.value(0)    # LOW = relay encendido  
        self._ventilador_on = True  
        return True  
  
    def apagar_ventilador(self):  
        self._relay1.value(1)  
        self._ventilador_on = False  
  
    def abrir_ventanas(self) -> bool:  
        if self._paro_activo: return False  
        self._servo_pwm.duty_u16(SERVO_ABIERTO)  
        self._ventanas_open = True  
        return True  
  
    def cerrar_ventanas(self):  
        self._servo_pwm.duty_u16(SERVO_CERRADO)  
        self._ventanas_open = False  
  
    def paro_emergencia(self):  
        self._relay1.value(1)  
        self._servo_pwm.duty_u16(SERVO_CERRADO)  
        self._ventanas_open = False  
        self._paro_activo = True  
  
    def reiniciar_circuito(self):  
        self._relay1.value(1)
```

```

        self._servo_pwm.duty_u16(SERVO_CERRADO)
        self._ventanas_open = False
        self._paro_activo = False

    def estado(self) -> dict:
        return {"ventilador": self._ventilador_on,
                "ventanas": self._ventanas_open,
                "paro": self._paro_activo}

```

9.4 sensores.py – Lectura de Sensores

Lee el DHT11 (temperatura, humedad, índice de calor, punto de rocío, confort térmico) y el MQ-135 (ADC crudo, voltaje, desviación estándar, porcentaje de calidad, CO₂ estimado, nivel de riesgo, recomendación de ventilación y alarma de hardware).

```

# sensores.py - Lectura de sensores ambientales (datos expandidos)
import dht
from machine import Pin, ADC
import time
from config import PIN_DHT11, PIN_MQ135_A, PIN_MQ135_D

class SensorDHT11:
    def __init__(self):
        self.sensor = dht.DHT11(Pin(PIN_DHT11))

    def leer(self) -> dict:
        try:
            self.sensor.measure()
            time.sleep_ms(200)
            t = self.sensor.temperature()
            h = self.sensor.humidity()
            ic = self._indice_calor(t, h)
            pr = self._punto_rocio(t, h)
            cf = self._confort_termico(t, h)
            return {"temperatura": t, "humedad": h,
                    "indice_calor": ic, "punto_rocio": pr,
                    "confort": cf, "error": False}
        except Exception as e:
            return {"temperatura": None, "humedad": None, "error": True}

class SensorMQ135:
    def __init__(self):
        self.adc = ADC(Pin(PIN_MQ135_A))
        self.adc.atten(ADC.ATTN_11DB)
        self.adc.width(ADC.WIDTH_12BIT)
        self.pin_digital = Pin(PIN_MQ135_D, Pin.IN)

    def leer(self) -> dict:
        muestras = [self.adc.read() for _ in range(10)]
        valor = sum(muestras) // len(muestras)

```

```

        alarma = not self.pin_digital.value()
        voltaje = round((valor / 4095.0) * 3.3, 3)
        porcentaje = max(0, min(100, int((1 - valor / 4095.0) * 100)))
        ppm_co2 = self._interpolar_ppm(valor, self._PPM_TABLA)
        # ... clasificacion y recomendacion segun valor ADC
        return {"valor_adc": valor, "voltaje_v": voltaje,
                "porcentaje": porcentaje, "ppm_co2": ppm_co2,
                "condicion": condicion, "nivel_riesgo": nivel_riesgo,
                "alarma_hw": alarma, "error": False}

```

9.5 sistema.py – Núcleo del Sistema

Coordina todos los módulos. Implementa el escaneo manual, el escaneo automático con toma de decisiones, el cambio de modo y la generación del estado completo para la interfaz web.

```

# sistema.py - Nucleo del Aula Inteligente
from config import TEMP_MAX_VENTILADOR, UMBRAL_GAS_PELIGROSO, INTERVALO_ESCASEO

class SistemaAula:
    def ejecutar_escaneo_automatico(self) -> dict:
        resultado = self.ejecutar_escaneo()
        temp, gas_adc, gas_alarma = resultado['temperatura'],
                                       resultado['gas_adc'],
                                       resultado['gas_alarma']

        # Logica de temperatura
        if temp is not None:
            if temp > TEMP_MAX_VENTILADOR:      # > 28 C
                self.actuadores.activar_ventilador()
            else:
                self.actuadores.apagar_ventilador()

        # Logica de calidad del aire
        if gas_adc > UMBRAL_GAS_PELIGROSO or gas_alarma: # > 2500 ADC
            self.actuadores.abrir_ventanas()
        else:
            self.actuadores.cerrar_ventanas()

        self._escribir_json_ia() # Actualizar datos_ia.json
        return resultado

```

9.6 json_ia.py – Generador de JSON para IA

Genera el archivo datos_ia.json con valores numéricos y descripciones textuales en español para que cualquier modelo de lenguaje (ChatGPT, Gemini, Claude, etc.) pueda analizar el estado del aula sin preprocesamiento adicional.

```

# json_ia.py - Generador de JSON de analisis para IA
RUTA_JSON_IA = '/datos_ia.json'

def generar_y_guardar(escaneo, estado_actuadores, modo, seg_boot, ...):

```

```

    documento = {
        "instruccion_para_ia": (
            "Eres un asistente de analisis ambiental. "
            "Analiza los datos de este JSON que corresponden al ultimo escaneo
"
            "del Aula Inteligente. Identifica si existen condiciones fuera de
rango, "
            "evalua el confort de los ocupantes, determina si los actuadores
estan "
            "respondiendo correctamente, y proporciona recomendaciones
concretas "
            "en espanol para mejorar el ambiente del aula."
        ),
        "temperatura_y_humedad": {
            "temperatura_celsius": {"valor": t, "descripcion": "..."}
            # ... todos los demas campos con valor + descripcion en espanol
        }
    }
    with open(RUTA_JSON_IA, 'w') as f:
        json.dump(documento, f)

```

9.7 servidor_http.py – Servidor HTTP Embebido

Implementa un servidor HTTP de un solo hilo con envío de HTML fragmentado (512 bytes por fragmento) para no saturar la heap de la ESP32. Las respuestas JSON siempre son pequeñas y se construyen directamente en memoria.

```

# servidor_http.py - Servidor HTTP embebido para la ESP32
TAMANO_FRAGMENTO = 512

class ServidorHTTP:
    def iniciar(self):
        addr = socket.getaddrinfo("0.0.0.0", self.puerto)[0][-1]
        self._sock = socket.socket()
        self._sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self._sock.bind(addr)
        self._sock.listen(2)
        self._sock.settimeout(0.5)

    def _servir_html(self, cliente):
        import pagina_web as pw
        longitud = len(pw.CABECERA_HTML.encode()) +
len(pw.CUERPO_HTML.encode())
        _enviar_cabecera_html(cliente, longitud)
        _enviar_fragmentos(cliente, pw.CABECERA_HTML) # 512 bytes a la vez
        _enviar_fragmentos(cliente, pw.CUERPO_HTML)

```

9.8 main.py – Punto de Entrada

Orquesta el arranque del sistema: inicializa el hardware, intenta conectarse al WiFi y lanza el servidor HTTP (modo manual) o el loop de escaneo periódico (modo automático sin WiFi).

```
# main.py - Punto de entrada del Aula Inteligente
from sistema import SistemaAula
from wifi_manager import conectar_wifi
from servidor_http import ServidorHTTP

aula = SistemaAula()
wifi_ok, ip = conectar_wifi()

if wifi_ok:
    aula.cambiar_modos("manual")
    servidor = ServidorHTTP(aula, puerto=80)
    servidor.iniciar()
    while True:
        servidor.atender_peticion()
        aula.tick_automatiko()
        gc.collect()
else:
    aula.cambiar_modos("automatico")
    aula.ejecutar_escaneo_automatiko()
    while True:
        aula.tick_automatiko()
```

10. Pruebas y Validación

10.1 Pruebas Funcionales por Componente

Componente	Prueba Realizada	Resultado
DHT11	Lectura continua durante 5 minutos con comparación contra termómetro de referencia.	Variación máxima de ± 2 °C respecto al termómetro. Consistente con la especificación del sensor.
MQ-135	Exposición a gas de aerosol para simular contaminación. Verificación de incremento de ADC y activación de alarma DO.	El ADC subió de ~800 (aire limpio) a >3000 en presencia de aerosol. La alarma DO se activó correctamente.
Relay + Ventilador	Activación y desactivación desde la interfaz web en modo manual. Verificación de la lógica invertida (LOW = encendido).	El ventilador enciende y apaga sin retardos. Lógica invertida del relay funcionando correctamente.
Servo SG90	Comandos de abrir (30°) y cerrar (0°) desde la web. Verificación de posiciones con transportador.	El servo alcanza las posiciones objetivo. duty_u16 calibrado al mecanismo físico de la maqueta.
Pantalla LCD	Visualización de temperatura, CO ₂ , modo y estado en las dos líneas de 16 caracteres.	La pantalla muestra la información correctamente, con relleno de espacios para limpiar caracteres residuales.
Servidor HTTP	Carga de la interfaz web desde dispositivos móvil, PC y tablet en la misma red.	La página carga en todos los dispositivos. El envío fragmentado de 512 bytes no agota la heap.
Ngrok	Acceso a la interfaz desde una red externa usando la URL pública de Ngrok.	La interfaz web responde correctamente desde Internet. Latencia adicional de ~80-200 ms por el túnel.
Modo Automático	Simulación de temperatura alta (>28 °C) y gas peligroso (ADC >2500).	El ventilador se activó automáticamente con temperatura alta. Las ventanas se abrieron con gas alto.
Paro de Emergencia	Activación del paro y verificación de que los actuadores no respondan hasta reiniciar.	Todo se apagó inmediatamente. Los botones de control quedaron deshabilitados hasta presionar Reiniciar.

10.2 Validación del Modelo JSON para IA

Escenario de Prueba	Descripción del JSON Enviado	Respuesta de ChatGPT
Aula Normal	T=23°C, H=55%, ADC=800, ventilador apagado, ventanas cerradas.	Estado NORMAL. Condiciones óptimas de aprendizaje. No se requiere ninguna acción.
Temperatura Alta	T=31°C, H=70%, ADC=1200, ventilador encendido automáticamente.	Estado ATENCIÓN. Temperatura sobre el límite confort. El ventilador respondió correctamente. Recomenda abrir ventanas si no baja en 5 minutos.
Calidad de Aire Peligrosa	T=25°C, H=60%, ADC=3100, alarma DO activa, ventanas abiertas.	Estado CRÍTICO. Concentración de gases peligrosa. El sistema abrió ventanas correctamente. Recomenda además evacuar si el ADC no baja en 2 min.
Paro de Emergencia	Todos los actuadores detenidos por paro activo.	Estado EMERGENCIA. El sistema está en paro de seguridad. Recomenda inspección física antes de reiniciar.

11. Documentación Técnica Complementaria

11.1 Configuración e Instalación

- Instalar MicroPython en la ESP32 usando esptool.py o Thonny IDE.
- Copiar todos los archivos .py a la raíz del sistema de archivos de la ESP32 (LittleFS).
- Configurar las credenciales WiFi en config.py (REDES_WIFI).
- Conectar los sensores y actuadores según el diagrama de conexiones (Sección 5.5).
- Encender la ESP32: boot.py apaga los actuadores → main.py arranca el sistema.
- Instalar Ngrok en una PC de la misma red: ngrok http [IP_LOCAL_ESP32]:80.
- Compartir la URL pública de Ngrok para acceso externo a la interfaz web.

11.2 Guía de Uso de la Interfaz Web

1. Acceder a la URL pública de Ngrok o a http://[IP_ESP32] desde la red local.
2. La interfaz carga el estado actual del sistema (modo, actuadores, último escaneo).
3. Seleccionar Modo Manual o Modo Automático según la necesidad.
4. En Modo Manual: usar los botones de control para encender/apagar el ventilador y abrir/cerrar ventanas.
5. Presionar 'Ejecutar Escaneo' para obtener datos actualizados de los sensores.
6. Para análisis con IA: ir a la ruta /datos_ia, descargar el JSON y adjuntarlo a ChatGPT.
7. En caso de emergencia: presionar PARO DE EMERGENCIA. Para reanudar: presionar Reiniciar Circuito.

11.3 Consideraciones de Seguridad

- El relay del ventilador usa lógica invertida (HIGH = apagado) para asegurar que, en caso de fallo del GPIO, el ventilador quede apagado y no encendido.
- boot.py garantiza que todos los actuadores arranquen en estado seguro antes de que main.py tome control.
- El paro de emergencia bloquea todos los actuadores y requiere acción manual explícita para reiniciar.
- El servidor HTTP tiene un timeout de 0.5 s por petición para no bloquear el loop principal indefinidamente.
- La contraseña WiFi se almacena en texto plano en config.py; en un despliegue real se recomienda usar almacenamiento seguro.

11.4 Posibles Expansiones Futuras

- Calentador PTC 110V: el firmware ya incluye la lógica completa para un segundo relay (PIN_RELAY2 = 27) con la función activar_calentador() y el umbral TEMP_MIN_CALENTADOR = 10 °C. Solo requiere agregar el hardware.

- RTC externo (DS3231): añadir marcas de tiempo reales al historial y al JSON de IA en lugar de segundos desde el boot.
- Almacenamiento en la nube: enviar el historial FIFO a Firebase o ThingSpeak mediante HTTP POST.
- Más sensores: PM2.5 para partículas finas, sensor de luz BH1750, sensor de ruido.
- Notificaciones push: integrar con Telegram Bot API o IFTTT para alertas automáticas por gas peligroso.
- Dashboard web avanzado: gráficas históricas de temperatura y CO₂ usando Chart.js en la interfaz web.

12. Referencias

Espressif Systems. (2024). ESP32 Technical Reference Manual. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

MicroPython Documentation. (2024). MicroPython – Python for microcontrollers. <https://docs.micropython.org/en/latest/>

Winsen Electronics. (2020). MQ-135 Gas Sensor Datasheet. <https://www.winsen-sensor.com/sensors/co2-sensor/mq135.html>

Aosong Electronics. (2021). DHT11 Humidity & Temperature Sensor Datasheet.

Tower Pro. (2023). SG90 Micro Servo Motor Datasheet. <http://www.towerpro.com.tw/product/sg90-7/>

Ngrok Inc. (2024). Ngrok Documentation. <https://ngrok.com/docs>

OpenAI. (2024). ChatGPT. <https://chat.openai.com>

American Psychological Association. (2020). Publication Manual of the American Psychological Association (7.^a ed.). <https://apastyle.apa.org>

Von Neumann, J. (1945). First Draft of a Report on the EDVAC. Moore School of Electrical Engineering, University of Pennsylvania.

Steadman, R. G. (1979). The assessment of sultriness. Part I: A temperature-humidity index based on human physiology and clothing science. *Journal of Applied Meteorology*, 18(7), 861-873.